MongoDB Worksheet

Subject 3 Worksheet

This activity involves creating and modifying a MongoDB database using MongoDB Compass and a NodeJS application with Mongoose.

Preliminary information:

- **MongoDB** - MongoDB is a NoSQL database that stores data in flexible JSON-like documents instead of tables.
- **Collection** - A collection is a group of related documents in MongoDB. It is similar to a table in SQL databases.
- **Document** - A document is a single record stored in MongoDB. It is written in JSON format and contains fields and values.
- **MongoDB Compass** - MongoDB Compass is a graphical user interface that allows you to view, create, and modify MongoDB databases and collections.

Step 1 - Create a Database using MongoDB Compass

Open MongoDB Compass and ensure you are connected to your cluster. Ask your trainer for help if you don't have a cluster set up.

Follow these steps:

1. Click the "Create Database" button.

2. Enter the following Database Name:
   student_database
3. Enter the following Collection Name:
   tasks
4. Click "Create Database".

You have now created a database and collection. You can create many collections per database, but we'll just stick to one.

Step 2 - Create Documents using MongoDB Compass

You will now manually insert documents.

1. Click on the "tasks" collection.
2. Click the "Add Data" then "Insert Document" button.
3. Replace the contents with the following:

```
{
 "title": "Complete worksheet",
 "description": "Finish the MongoDB activity",
 "dueDate": "2026-03-10",
 "completed": false
}
```

1. Click "Insert".
2. Insert another document:

```
{
 "title": "Study MongoDB",
 "description": "Review collections and documents",
 "dueDate": "2026-03-12",
 "completed": false
}
```

*Investigate with the **_id** is. How does is relate to Primary Keys used in SQL databases?*

You should now see two documents in your collection. We have manually been adding data into our MongoDB database. However, in reality we typically only use a database through a backend server. We will now explore doing this in NodeJS.

Delete the database for now by clicking on the rubbish button.

Step 3 - Create a NodeJS Project

Open Visual Studio Code.

Open a new folder called:

mongo_activity

Open a terminal and run the following commands:

npm init -y

Then install the required packages:

npm install mongoose

Command	Explanation
npm init -y	Quickly creates a new Node.js project with default settings.
npm install mongoose	Adds the Mongoose library so your Node.js app can work with MongoDB.

Step 4 - Create the Application File

Create a new file called:

app.js

Paste the following code:

```
1  const mongoose = require("mongoose");
2
3  mongoose.connect("YOUR_CONNECTION_STRING_HERE/student_database")
4  .then(
5      () => console.log("Connected to MongoDB")
6  );
7
8  const taskSchema = new mongoose.Schema({
9      title: {type: String, required: true},
10     description: {type: String, required: true},
11     dueDate: {type: Date, required: true},
12     dateCreated: {type: Date, default: Date.now},
13     completed: {type: Boolean, default: false}
14 });
15
16 const Task = mongoose.model("Task", taskSchema);
17
18 const newTask = new Task({
19     title: "Created from NodeJS",
20     description: "This task was created using Mongoose",
21     dueDate: new Date("2026-03-20")
22 });
23
24 newTask.save().then(
25     () => {
26         console.log("Added new entry into the database");
27         mongoose.disconnect();
28     }
29 );
```

Lines	Explanation
1	Imports mongoose package to be used later on.
3-6	Connects to MongoDB. Executes the arrow function inside .then() if the connection is successful. Also creates the database “student_database” automatically if it doesn’t exist.
8-14	Creates a schema for a document structure to be used with MongoDB. Remember, documents are JSON files within MongoDB. This schema makes it easier for us to have a consistent JSON structure.
16	Creates the collection, which mongoose calls a “model”. Also creates the “tasks” collection if it doesn’t exist.
18-22	Creates a new document with the given field data in the “tasks” collection.
24-29	Saves the new document so it actually gets inserted into the database. Executes the given arrow function if it successfully inserted the data.

Step 5 - Run the Application

In the terminal, run:

node app.js

You should see:

Connected to MongoDB
Added new intry into th database

Step 6 — Verify in MongoDB Compass

Return to MongoDB Compass.

1. Click the tasks collection.
2. Click "Find" to refresh.

You should see the new document created by NodeJS.

Step 7 — Modify the Application to Read Data

We are now going to investigate further adding CRUD operations in NodeJS. We have just investigated making a Create option in NodeJS, so we'll look at a Read operation.

Remember, CRUD operations stand for Create, Read, Update and Delete. It sums up all the ways we can interact with a database.

Replace lines 24 to 29 of your code with:

```
24  Task.find().then(
25      tasks => {
26          console.log(tasks);
27          mongoose.disconnect();
28      }
29  ).catch(
30      err => console.log(err)
31  );
```

Run:

node app.js

You should see all tasks printed in the terminal. The *Task.find()* function returns all document entries for the respective collection. The arrow function in *then()* is ran after the find function returns all the entries. The arrow function in *catch()* is ran if any errors occur.

Step 8 - Modify the Application to Update Data

Now replace lines 24 to 31 of your code with:

```
24 Task.updateOne(
25     { title: "Created from NodeJS" },
26     { completed: true }
27 )
28 .then(
29     () => {
30         console.log("Task updated");
31         mongoose.disconnect();
32     }
33 )
34 .catch(err => console.log(err));
```

Run:

node app.js

If you refresh your Mongo Compass view of the tasks collection, you'll see an updated document.

The *Task.updateOne()* function takes in two parameters. The first parameter is a filter; the first document with a match to the filter gets updated. The second parameter is what we update for the document.

Step 9 - Modify the Application to Delete Data

Now finally, replaces lines 24 to 34 with the following code:

```
24 Task.deleteOne({ title: "Created from NodeJS" })
25 .then(
26     () => {
27         console.log("Task deleted");
28         mongoose.disconnect();
29     }
30 )
31 .catch(err => console.log(err));
```

Run the program and verify the document has been deleted.

The *Task.deleteOne()* function takes in one parameter. It is a filter; the first document that matches the filter gets deleted.

For full comprehensive documentation on how to fully utilise mongoose with MongoDB, refer to it's documentation page: https://mongoosejs.com/docs/

End of Activity